Integer Arithmetic: Well-Ordering, Euclidean Algorithm, and the Euler Function

N-20221030

§1.1 The well-ordering principle

The note begins with a concise list of basic facts about integers. The first is the well-ordering principle:

Theorem 1.1.1 (Well-ordering of $\mathbb{Z}_+$)
Every nonempty subset $A \subseteq \mathbb{Z}_+$ has a least element.

This statement looks elementary, but it is the engine behind many proofs in number theory. It is the reason the Euclidean algorithm terminates, the reason minimal-counterexample arguments work, and one of the standard formulations equivalent to induction.

§1.2 Divisibility, gcd, and lcm

For integers a, b, with $a \neq 0$, we say

$$a \mid b$$

if there exists $c \in \mathbb{Z}$ such that

$$b = ac.$$

The greatest common divisor of nonzero integers a, b is the positive integer d such that

$$d \mid a, \qquad d \mid b,$$

and if $e \mid a$ and $e \mid b$, then $e \mid d$. We write

$$d = (a, b).$$

If $(a, b) = 1$, then a and b are relatively prime.

Similarly, the least common multiple l is the positive integer such that

$$a \mid l, \qquad b \mid l,$$

and every common multiple of a, b is a multiple of l. For positive a, b,

$$(a, b)[a, b] = ab.$$

The relation is best understood prime-by-prime: gcd takes the smaller exponent, lcm takes the larger exponent.

§1.3 Division algorithm

The division algorithm says that for $a, b \in \mathbb{Z}$, $b \neq 0$, there exist unique integers q, r such that

$$a = qb + r, \qquad 0 \leq r < |b|.$$

Here q is the quotient and r is the remainder. This is not merely long division. It is the formal foundation of modular arithmetic and the Euclidean algorithm.

§1.4 Euclidean algorithm

Starting with $a, b \neq 0$, repeatedly divide:

$$\begin{aligned} a &= q_0 b + r_0, \\ b &= q_1 r_0 + r_1, \\ r_0 &= q_2 r_1 + r_2, \\ &\vdots \\ r_{n-2} &= q_n r_{n-1} + r_n, \\ r_{n-1} &= q_{n+1} r_n. \end{aligned}$$

The last nonzero remainder r_n is the gcd of a, b.

The reason is the invariant

$$(a, b) = (b, r_0) = (r_0, r_1) = \cdots = (r_{n-1}, r_n) = r_n.$$

At each step, replacing (a, b) by $(b, a - qb)$ does not change the set of common divisors. This is the real idea; the numerical algorithm is just repeated use of this invariant.

§1.5 Example

The note gives a Euclidean-algorithm example such as

$$57970 \quad \text{and} \quad 10353.$$

The computation proceeds by repeated division until the remainder becomes zero. The last nonzero remainder is the gcd. The same backward substitutions produce integers x, y such that

$$ax + by = (a, b).$$

This is Bézout's identity.

§1.6 Fundamental theorem of arithmetic

Theorem 1.6.1 (Fundamental theorem of arithmetic)
Every integer $n > 1$ can be written as a product of primes, and this factorization is unique up to the order of the factors.

Every integer $n > 1$ factors uniquely into primes:

$$n = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_k^{\alpha_k}.$$

The note presents this as an essential property of integers. Uniqueness is what makes gcd, lcm, and the Euler function computable by exponents.

For example, if

$$a = \prod p^{\alpha_p}, \qquad b = \prod p^{\beta_p},$$

then

$$(a, b) = \prod p^{\min(\alpha_p, \beta_p)}, \qquad [a, b] = \prod p^{\max(\alpha_p, \beta_p)}.$$

§1.7 The Euler phi function

The Euler function $\varphi(n)$ counts the positive integers not exceeding n that are coprime to n. If

$$n = p_1^{\alpha_1} \cdots p_k^{\alpha_k},$$

then

$$\varphi(n) = n \left(1 - \frac{1}{p_1}\right) \cdots \left(1 - \frac{1}{p_k}\right).$$

This follows by inclusion–exclusion: remove multiples of each prime divisor of n.

For a prime power,

$$\varphi(p^a) = p^a - p^{a-1}.$$

For a prime p,

$$\varphi(p) = p - 1.$$

§1.8 The broader lesson

This note is a compact bridge from elementary arithmetic to algebra. The Euclidean algorithm is not just a computation; it is an algorithmic proof that $\mathbb{Z}$ is a Euclidean domain. Bézout identity says ideals in $\mathbb{Z}$ are principal. Unique factorization says primes control divisibility. The Euler function prepares the way for the multiplicative group $(\mathbb{Z}/n\mathbb{Z})^\times$, where number theory becomes group theory.

§1.9 Well-ordering and Euclidean domains

The well-ordering principle makes the division algorithm possible in $\mathbb{Z}$. The Euclidean algorithm works because each step strictly decreases a nonnegative remainder. This pattern generalizes to Euclidean domains.

§1.10 Bezout and principal ideals

The gcd of a and b is the positive generator of the ideal

$$(a) + (b) = \{ax + by : x, y \in \mathbb{Z}\}.$$

Bezout's identity says the gcd is an integer linear combination of a and b.

§1.11 Euler function and unit groups

Euler's function counts units modulo n:

$$\varphi(n) = |(\mathbb{Z}/n\mathbb{Z})^\times|.$$

Euler's theorem is Lagrange's theorem in this finite group:

$$a^{\varphi(n)} \equiv 1 \pmod{n}$$

whenever $\gcd(a, n) = 1$.

§1.12 Euclidean arithmetic as ideals and unit groups

Integer arithmetic is organized by order, ideals, and groups. Well-ordering drives division, Euclidean algorithms compute ideal generators, unique factorization gives valuations, and Euler's theorem is group theory modulo n.

§1.13 Euclidean domains

The Euclidean algorithm works in $\mathbb{Z}$ because there is a division process with decreasing remainder. A Euclidean domain is an integral domain R with a size function N such that for $a, b \in R$, $b \neq 0$, one can write

$$a = qb + r, \qquad r = 0 \text{ or } N(r) < N(b).$$

The integers and polynomial rings $k[x]$ over a field are Euclidean domains.

In every Euclidean domain, the Euclidean algorithm produces greatest common divisors and Bezout identities. Thus the elementary gcd algorithm is one example of a general algebraic mechanism.

§1.14 PID and Bezout domains

A principal ideal domain is an integral domain in which every ideal is generated by one element. In a PID, gcds exist because

$$(a) + (b) = (d)$$

for some d, and that d is a greatest common divisor of a and b.

Bezout's identity

$$ax + by = d$$

means precisely that $d \in (a) + (b)$ and generates the sum ideal. This reframes gcd as an ideal-theoretic object.

§1.15 Euler phi as unit counting

Euler's function counts units in a quotient ring:

$$\varphi(n) = |(\mathbb{Z}/n\mathbb{Z})^\times|.$$

If

$$n = \prod_i p_i^{e_i},$$

then the Chinese remainder theorem gives

$$(\mathbb{Z}/n\mathbb{Z})^\times \cong \prod_i (\mathbb{Z}/p_i^{e_i}\mathbb{Z})^\times.$$

Therefore

$$\varphi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right).$$

The familiar formula is a group-size computation.

§1.16 Multiplicative functions

A function $f(n)$ is multiplicative if $f(mn) = f(m)f(n)$ whenever $\gcd(m, n) = 1$. Euler's phi, the divisor-counting function $\tau(n)$, and the sum-of-divisors function $\sigma(n)$ are multiplicative.

The reason is again Chinese remaindering or prime factorization: arithmetic data often decomposes independently over primes. This is the elementary beginning of Euler products and analytic number theory.

§1.17 Euclidean algorithm and ideals

The Euclidean algorithm does more than compute a gcd. It constructs coefficients x, y such that

$$ax + by = \gcd(a, b).$$

The back-substitution process is a constructive proof that the ideal (a, b) is principal.

This construction also gives modular inverses. If $\gcd(a, n) = 1$, then

$$ax + ny = 1,$$

so x is the inverse of a modulo n. The original algorithmic procedure is therefore the computational heart of quotient-ring arithmetic.

§1.18 Euler's theorem as group theory

Euler's theorem

$$a^{\varphi(n)} \equiv 1 \pmod{n}$$

for $\gcd(a, n) = 1$ follows from Lagrange's theorem in the finite group $(\mathbb{Z}/n\mathbb{Z})^\times$. The elementary proof by reduced residue systems is already a group action proof: multiplication by a permutes the reduced residues.

This viewpoint clarifies why the exponent $\varphi(n)$ appears: it is the order of the group of units.